

山东大学



网络安全创新创业实践课实验报告

对称密码算法的软件实现与优化

第 41 组

2026 年 7 月 26 日

本实验由小组成员共同完成。各成员具体分工如表 1 所示。

表 1: 小组成员及分工

姓名	学号	班级	负责模块	对应章节
姜良萍 (组长)	202300460098	网安 1 班	负责实验总体方案与项目框架设计; 完成 AES-128 字节级基础加解密、密钥扩展、统一多分组接口及 T-table 四表优化; 组织 CMake 构建、标准测试向量和随机交叉验证; 负责各模块集成、实验结果核对及报告整体校验。	第 1 章; 第 2 章; 第 3.1 节; 第 7.1 节; 第 9.2 节; 第 10.1 节; 第 11 章
王奕文	202300460056	密码 1 班	负责 GCM 模式的实现与分析; 完成便携式 GHASH、PCLMULQDQ 有限域乘法及后端组合设计; 验证 AAD、不同长度 IV、截断 Tag 和篡改拒绝功能, 并整理认证失败处理与定时安全相关分析。	第 5 章; 第 7.2 节; 第 9.4 节; 第 10.2 节
李雨晴	202300460025	网安 2 班	负责 AVX2 gather/shuffle 八路实现和 CTR 模式优化; 完成跨分组数据布局、字节序转换、并行查表、批量计数器生成和 SIMD 密钥流异或; 验证计数器进位、耗尽保护及尾部分组处理, 并整理相关性能结果。	第 3.2 节; 第 4 章; 第 7.3 节; 第 9.3 节; 第 10.3 节
曾静雯	202300460045	网安 1 班	负责 AES-NI 8/4/1 路加解密和 XTS 模式优化; 完成批量 tweak、XEX 白化及 Ciphertext Stealing; 组织固定 CPU 核心的重复性能测试, 统计中位数并绘制性能图表, 整理总体结果、实验局限性和附录数据。	第 3.3 节; 第 6 章; 第 8 章; 第 9.1、9.5 节; 第 10.4 节; 附录 A

目录

1 实验目标与总体设计	1
1.1 实验目标	1
1.2 总体架构	1
2 AES-128 算法与基础实现	2
2.1 算法结构	2
2.2 密钥扩展	3
2.3 字节级加密实现	4
3 AES 核心的软件优化设计	6
3.1 T-table 实现	6
3.1.1 优化原理	6
3.1.2 状态表示与轮函数	6
3.2 AVX2 gather/shuffle 八路实现	8
3.2.1 数据布局	8
3.2.2 向量查表轮函数	9
3.2.3 尾部处理	11
3.3 AES-NI 专用指令实现	11
3.3.1 单分组轮函数	11
3.3.2 多分组交错执行	13
3.3.3 运行时检测与回退	14
4 CTR 工作模式实现与优化	15
4.1 模式原理	15
4.2 实现与优化	15
5 GCM 工作模式实现与优化	16
5.1 模式原理	16
5.2 后端设计	16
6 XTS 工作模式实现与优化	17
6.1 模式原理	17
6.2 tweak 更新	17
6.3 批量处理	18
7 实验验证与性能测试设计	18
7.1 AES 核心正确性验证	18
7.2 工作模式正确性验证	19

7.3	硬件指令验证	19
7.4	实验环境	20
7.5	性能测试设计	20
7.6	性能指标与统计方法	21
8	实验结果与分析	21
8.1	1 MiB 输入下的总体结果	21
8.2	AES 核心性能分析	22
8.3	CTR 性能分析	22
8.4	GCM 性能分析	23
8.5	XTS 性能分析	23
9	安全性与工程权衡	23
9.1	T-table 与缓存侧信道	23
9.2	常数时间与认证失败处理	23
9.3	输入边界与状态管理	24
9.4	实验实现的局限性	24
10	结论	24
A	完整性能中位数数据	25
A.1	AES 核心	25
A.2	CTR	25
A.3	GCM	26
A.4	XTS	26

1 实验目标与总体设计

1.1 实验目标

课程作业要求从基本实现出发提高对称密码的软件执行效率，覆盖 T-table、shuffle 以及至少两类现代指令集方法，并在分组密码核心之上完成 CTR、GCM 和 XTS 工作模式的优化。综合实现难度、标准支持和测试便利性，本实验选择 AES-128，主要完成以下任务：

1. 实现符合 AES-128 标准流程的基础版本；
2. 实现 T-table、AVX2 gather/shuffle 和 AES-NI 三种优化后端；
3. 在统一接口上实现 CTR、GCM 和 XTS 工作模式；
4. 使用 PCLMULQDQ 优化 GCM 中的 GHASH，使用批量 tweak 优化 XTS；
5. 通过标准向量、随机交叉验证和重复性能测试评价实现效果。

1.2 总体架构

程序采用分层设计，将 AES 核心实现与工作模式分离。底层负责密钥扩展和单分组、多分组加解密；上层 CTR、GCM 和 XTS 通过统一接口调用不同的 AES 后端。

如图 1 所示，AES 核心包含四种实现：

- Reference：按字节实现标准轮函数，作为正确性与性能基准；
- T-table：将 SubBytes、ShiftRows 和 MixColumns 合并为查表操作；
- AVX2：使用 gather 和 shuffle 实现八分组并行；
- AES-NI：使用专用 AES 指令实现 8/4/1 路加解密。

工作模式层在上述核心基础上继续优化：

- CTR：批量生成计数器并使用 SIMD 完成密钥流异或；
- GCM：在 CTR 基础上加入 GHASH，并使用 PCLMULQDQ 加速有限域乘法；
- XTS：预先生成多个 tweak，并结合 AES-NI、XEX 和 ciphertext stealing。

程序运行时检测 AVX2、AES-NI 和 PCLMULQDQ 支持情况。当对应指令集不可用时，自动回退到可用的软件路径。

不同 AES 后端通过统一的多分组接口向工作模式提供服务：

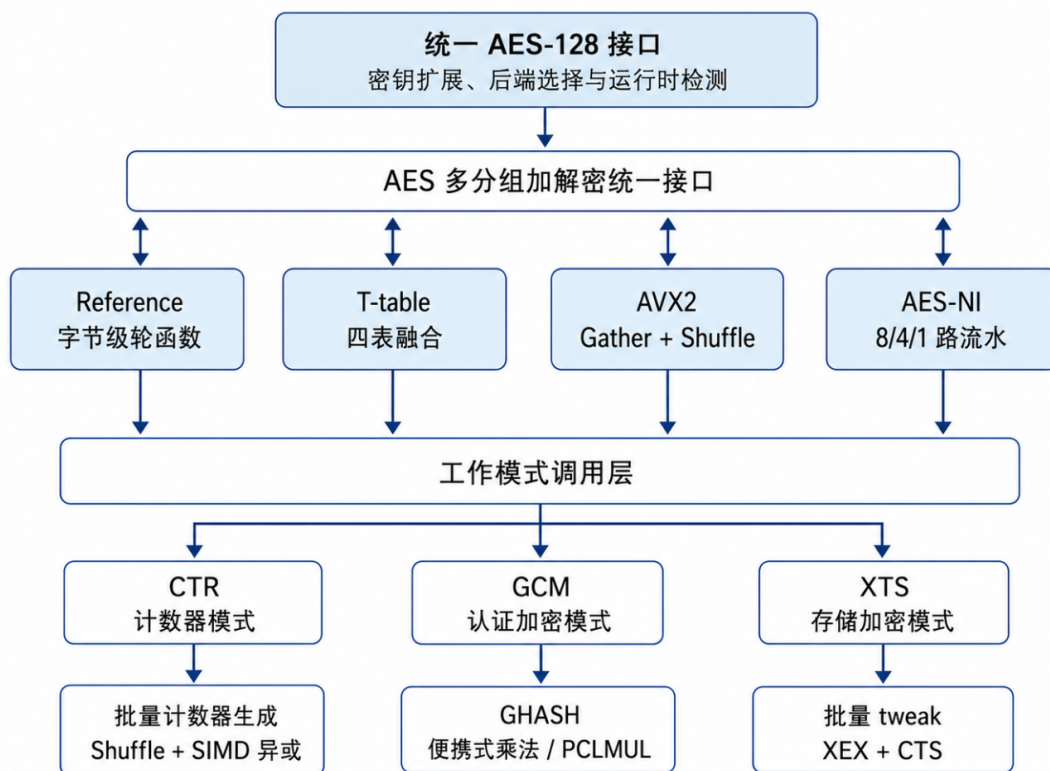


图 1: AES-128 核心后端与工作模式优化的分层架构

Listing 1: AES 多分组后端统一接口

```

1 typedef void (*aes_encrypt_blocks_fn)(
2     const aes128_key_t *ctx,
3     const uint8_t *in,
4     uint8_t *out,
5     size_t blocks
6 );
    
```

该设计使工作模式逻辑与 AES 核心实现相互独立，也便于在相同输入和测试环境下比较不同后端的性能。

后续仅列出能够体现实现思路的关键代码，S 盒、T-table 常量、重复轮展开和完整边界检查不在正文中全部展开。

2 AES-128 算法与基础实现

2.1 算法结构

AES-128 的分组长度和密钥长度均为 128 bit。一个分组包含 16 字节，并按列优先顺序映射为 4×4 状态矩阵：

$$S = \begin{bmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{bmatrix}. \quad (1)$$

AES-128 首先执行一次 AddRoundKey, 随后执行 9 个普通轮和 1 个末轮。普通轮为:

$$X^{(r+1)} = \text{AddRoundKey}(\text{MixColumns}(\text{ShiftRows}(\text{SubBytes}(X^{(r)}))), K^{(r)}). \quad (2)$$

末轮省略 MixColumns。解密执行对应的 InvShiftRows、InvSubBytes、AddRoundKey 和 InvMixColumns。

2.2 密钥扩展

AES-128 将 16 字节原始密钥扩展为 11 组轮密钥, 共 176 字节。扩展过程中依次执行 RotWord、SubWord、Rcon 异或以及与前一轮对应字的异或。

Listing 2: AES-128 密钥扩展关键逻辑

```
1 memcpy(ctx->round_keys, key, AES128_KEY_SIZE);
2
3 size_t generated = AES128_KEY_SIZE;
4 unsigned rcon_index = 0;
5
6 while (generated < AES128_EXPANDED_KEY_SIZE) {
7     uint8_t temp[4];
8     memcpy(temp, &ctx->round_keys[generated - 4], 4);
9
10    if ((generated % AES128_KEY_SIZE) == 0U) {
11        const uint8_t first = temp[0];
12
13        temp[0] = sbox[temp[1]];
14        temp[1] = sbox[temp[2]];
15        temp[2] = sbox[temp[3]];
16        temp[3] = sbox[first];
17
18        temp[0] ^= rcon[rcon_index++];
19    }
20}
```

```
21     for (size_t i = 0; i < 4; ++i) {
22         ctx->round_keys[generated] =
23             ctx->round_keys[
24                 generated - AES128_KEY_SIZE
25             ] ^ temp[i];
26
27         ++generated;
28     }
29 }
```

扩展后的轮密钥由所有 AES 后端共享。Reference 版本按字节读取，T-table 版本按 32 bit 字读取，AES-NI 版本直接加载为 128 bit 向量。

2.3 字节级加密实现

基础版本使用 16 字节数组保存状态，并分别调用 SubBytes、ShiftRows、MixColumns 和 AddRoundKey。其代码结构与 AES-128 标准轮流程保持一致，便于检查各轮变换和验证后续优化实现。

Listing 3: AES-128 字节级加密核心

```
1  uint8_t state[16];
2  memcpy(state, in, 16);
3
4  add_round_key(state, ctx->round_keys);
5
6  for (size_t round = 1;
7       round < AES128_ROUNDS;
8       ++round) {
9
10     sub_bytes(state);
11     shift_rows(state);
12     mix_columns(state);
13
14     add_round_key(
15         state,
16         &ctx->round_keys[
17             round * AES128_BLOCK_SIZE
18         ]
19     );
20 }
```



```
21
22 /* Final round without MixColumns. */
23 sub_bytes(state);
24 shift_rows(state);
25
26 add_round_key(
27     state,
28     &ctx->round_keys[
29         AES128_ROUNDS * AES128_BLOCK_SIZE
30     ]
31 );
32
33 memcpy(out, state, 16);
```

解密从最后一组轮密钥开始，并按相反顺序执行逆变换：

Listing 4: AES-128 字节级解密核心

```
1 memcpy(state, in, 16);
2
3 add_round_key(
4     state,
5     &ctx->round_keys[
6         AES128_ROUNDS * AES128_BLOCK_SIZE
7     ]
8 );
9
10 for (size_t round = AES128_ROUNDS - 1;
11     round > 0;
12     --round) {
13
14     inv_shift_rows(state);
15     inv_sub_bytes(state);
16
17     add_round_key(
18         state,
19         &ctx->round_keys[
20             round * AES128_BLOCK_SIZE
21         ]
22     );
23 }
```

```
24     inv_mix_columns(state);
25 }
26
27 inv_shift_rows(state);
28 inv_sub_bytes(state);
29 add_round_key(state, ctx->round_keys);
```

该版本主要作为标准流程实现、正确性基准和性能对照。由于 S 盒访问地址依赖中间状态，其缓存访问模式可能与密钥相关，因此不具备针对缓存计时攻击的防护能力。

3 AES 核心的软件优化设计

3.1 T-table 实现

3.1.1 优化原理

基础版本在每个普通轮中分别执行 SubBytes、ShiftRows 和 MixColumns。T-table 将上述三个变换预先组合为四张 32 bit 查找表，使普通轮主要由查表、异或和轮密钥加构成。

设 $S(b)$ 为字节 b 经 AES S 盒变换后的结果，则第一张表可表示为

$$T_0[b] = \begin{bmatrix} 2S(b) \\ S(b) \\ S(b) \\ 3S(b) \end{bmatrix}, \quad (3)$$

其余三张表由字节位置循环旋转得到。若状态表示为四个大端 32 bit 字 s_0, s_1, s_2, s_3 ，则普通轮的第一个输出字为

$$t_0 = T_0[s_0^{31:24}] \oplus T_1[s_1^{23:16}] \oplus T_2[s_2^{15:8}] \oplus T_3[s_3^{7:0}] \oplus K_0^{(r)}. \quad (4)$$

其余三个输出字通过轮换输入状态字获得。

3.1.2 状态表示与轮函数

实现中首先将输入分组拆分为四个大端 32 bit 状态字，并完成初始轮密钥加：

Listing 5: T-table 状态初始化

```
1 uint32_t s0 = load_be32(in + 0)
2     ^ round_key_word(ctx, 0);
3 uint32_t s1 = load_be32(in + 4)
4     ^ round_key_word(ctx, 1);
```

```
5 uint32_t s2 = load_be32(in + 8)
6         ^ round_key_word(ctx, 2);
7 uint32_t s3 = load_be32(in + 12)
8         ^ round_key_word(ctx, 3);
```

普通轮直接使用四张表计算新的状态字。下面保留实际实现中第一个输出字的计算，其余三个输出字采用相同的轮换方式：

Listing 6: T-table 普通轮关键计算

```
1 for (size_t round = 1;
2     round < AES128_ROUNDS;
3     ++round) {
4
5     const size_t k = round * 4U;
6
7     const uint32_t t0 =
8         aes128_te0[s0 >> 24]
9         ^ aes128_te1[(s1 >> 16) & UINT32_C(0xff)]
10        ^ aes128_te2[(s2 >> 8) & UINT32_C(0xff)]
11        ^ aes128_te3[s3 & UINT32_C(0xff)]
12        ^ round_key_word(ctx, k + 0U);
13
14    const uint32_t t1 =
15        aes128_te0[s1 >> 24]
16        ^ aes128_te1[(s2 >> 16) & UINT32_C(0xff)]
17        ^ aes128_te2[(s3 >> 8) & UINT32_C(0xff)]
18        ^ aes128_te3[s0 & UINT32_C(0xff)]
19        ^ round_key_word(ctx, k + 1U);
20
21    /* t2 and t3 use the same rotation rule. */
22
23    s0 = t0;
24    s1 = t1;
25    s2 = t2;
26    s3 = t3;
27 }
```

由于 AES 末轮省略 MixColumns，因此末轮不能继续使用包含列混合结果的 T-table。实现中重新使用 S 盒提取四个输出字节，并完成 ShiftRows 和 AddRoundKey：

Listing 7: T-table 末轮关键计算

```
1 const uint32_t t0 =
2     ((uint32_t)sbox_ttable[s0 >> 24] << 24)
3     ^ ((uint32_t)sbox_ttable[
4         (s1 >> 16) & UINT32_C(0xff)] << 16)
5     ^ ((uint32_t)sbox_ttable[
6         (s2 >> 8) & UINT32_C(0xff)] << 8)
7     ^ (uint32_t)sbox_ttable[
8         s3 & UINT32_C(0xff)]
9     ^ round_key_word(ctx, 40);
```

四张表共占用约 4 KiB。该方法减少了有限域乘法和字节级操作，但查表地址仍由中间状态决定，因此可能暴露缓存访问模式。

3.2 AVX2 gather/shuffle 八路实现

3.2.1 数据布局

AVX2 版本采用跨分组的 Structure of Arrays 布局，将 8 个分组的同一 32 bit 状态字放入一个 YMM 寄存器：

$$S_0 = [s_0^{(0)}, s_0^{(1)}, \dots, s_0^{(7)}]. \quad (5)$$

同理构造 S_1, S_2, S_3 。实际实现中，输入数据的处理分为两个步骤：

1. 使用 `_mm256_i32gather_epi32` 从 8 个分组中收集相同位置的 32 bit 状态字；
2. 使用 `_mm256_shuffle_epi8` 对每个 32 bit 字执行大端与主机字节序转换。

因此，shuffle 在该实现中主要用于数据装配和字节序转换，S 盒及 MixColumns 仍通过 T-table 完成。

Listing 8: AVX2 输入装配与字节序转换

```
1 const __m256i block_offsets =
2     _mm256_setr_epi32(
3         0, 16, 32, 48, 64, 80, 96, 112
4     );
5
6 __m256i s0 = _mm256_i32gather_epi32(
7     (const int*)(const void*)(in + 0),
8     block_offsets,
9     1
```

```
10 );  
11  
12 __m256i s1 = _mm256_i32gather_epi32(  
13     (const int *) (const void *) (in + 4),  
14     block_offsets,  
15     1  
16 );  
17  
18 s0 = byte_swap_words(s0);  
19 s1 = byte_swap_words(s1);
```

其中，字节交换函数使用 `_mm256_shuffle_epi8`:

Listing 9: AVX2 字节序转换

```
1 const __m256i mask = _mm256_setr_epi8(  
2     3, 2, 1, 0,  7, 6, 5, 4,  
3     11,10, 9, 8, 15,14,13,12,  
4     3, 2, 1, 0,  7, 6, 5, 4,  
5     11,10, 9, 8, 15,14,13,12  
6 );  
7  
8 return _mm256_shuffle_epi8(value, mask);
```

3.2.2 向量查表轮函数

每个 YMM 寄存器包含 8 个 32 bit 状态字。轮函数首先并行提取四个字节索引，然后分别从四张表中执行 gather:

Listing 10: AVX2 gather 并行查表核心

```
1 const __m256i mask_ff =  
2     _mm256_set1_epi32(0xff);  
3  
4 const __m256i i0 =  
5     _mm256_srli_epi32(a, 24);  
6  
7 const __m256i i1 =  
8     _mm256_and_si256(  
9         _mm256_srli_epi32(b, 16),  
10        mask_ff  
11    );
```

```
12
13 const __m256i i2 =
14     _mm256_and_si256(
15         _mm256_srli_epi32(c, 8),
16         mask_ff
17     );
18
19 const __m256i i3 =
20     _mm256_and_si256(d, mask_ff);
21
22 __m256i result =
23     gather_table(aes128_te0, i0);
24
25 result = _mm256_xor_si256(
26     result,
27     gather_table(aes128_te1, i1)
28 );
29
30 result = _mm256_xor_si256(
31     result,
32     gather_table(aes128_te2, i2)
33 );
34
35 result = _mm256_xor_si256(
36     result,
37     gather_table(aes128_te3, i3)
38 );
39
40 result = _mm256_xor_si256(
41     result,
42     _mm256_set1_epi32((int)round_key)
43 );
```

末轮由独立的 `final_round_word` 完成。由于末轮没有 MixColumns，程序从 Te0 表项中提取 S 盒字节，再根据 ShiftRows 的顺序重新组合输出字。

完成八分组加密后，各 YMM lane 中的结果被重新分散到 8 个连续输出分组，并调用 `_mm256_zeroupper`，减少后续执行 SSE 指令时可能产生的 AVX-SSE 状态切换开销。

3.2.3 尾部处理

AVX2 后端只在分组数不少于 8 时进入八路路径，剩余分组回退到标量 T-table:

Listing 11: AVX2 八路处理与尾部回退

```
1 if (aes128_avx2_supported()) {
2     while (blocks >= 8U) {
3         encrypt_eight_blocks_avx2(
4             ctx, in, out);
5
6         in      += 8U * AES128_BLOCK_SIZE;
7         out     += 8U * AES128_BLOCK_SIZE;
8         blocks -= 8U;
9     }
10 }
11
12 if (blocks != 0U) {
13     aes128_encrypt_blocks_ttable(
14         ctx, in, out, blocks);
15 }
```

因此，16 B 和 64 B 输入不会进入八路路径；输入达到 128 B 后，AVX2 版本才开始完整利用八路并行。

3.3 AES-NI 专用指令实现

3.3.1 单分组轮函数

AES-NI 使用专用指令完成 AES 轮变换。加密普通轮使用 `_mm_aesenc_si128`，末轮使用 `_mm_aesenclast_si128`:

Listing 12: AES-NI 单状态加密核心

```
1 static __m128i encrypt_state(
2     __m128i state,
3     const aes128_key_t *ctx
4 ) {
5     state = _mm_xor_si128(
6         state,
7         load_round_key(ctx, 0)
8     );
9
10     for (size_t round = 1;
```

```
11     round < AES128_ROUNDS;  
12     ++round) {  
13  
14     state = _mm_aesenc_si128(  
15         state,  
16         load_round_key(ctx, round)  
17     );  
18 }  
19  
20 return _mm_aesenc_si128(  
21     state,  
22     load_round_key(  
23         ctx, AES128_ROUNDS  
24     )  
25 );  
26 }
```

解密普通轮使用 `_mm_aesdec_si128`。由于程序存储的是加密轮密钥，中间解密轮密钥在使用前通过 `_mm_aesimc_si128` 转换：

Listing 13: AES-NI 单状态解密核心

```
1 static __m128i decrypt_state(  
2     __m128i state,  
3     const aes128_key_t *ctx  
4 ) {  
5     state = _mm_xor_si128(  
6         state,  
7         load_round_key(  
8             ctx, AES128_ROUNDS  
9         )  
10    );  
11  
12    for (size_t round =  
13        AES128_ROUNDS - 1U;  
14        round > 0U;  
15        --round) {  
16  
17        const __m128i inverse_key =  
18            _mm_aesimc_si128(  
19                load_round_key(ctx, round)
```



```
20         );
21
22         state = _mm_aesdec_si128(
23             state,
24             inverse_key
25         );
26     }
27
28     return _mm_aesdeclast_si128(
29         state,
30         load_round_key(ctx, 0)
31     );
32 }
```

3.3.2 多分组交错执行

单个状态的相邻 AES 轮之间存在数据依赖。实际实现分别维护 1 个、4 个或 8 个独立的 XMM 状态，并在每一轮使用相同轮密钥依次更新这些状态。例如八路实现每轮执行：

Listing 14: AES-NI 八分组交错执行

```
1 rk = load_round_key(ctx, round);
2
3 s0 = _mm_aesenc_si128(s0, rk);
4 s1 = _mm_aesenc_si128(s1, rk);
5 s2 = _mm_aesenc_si128(s2, rk);
6 s3 = _mm_aesenc_si128(s3, rk);
7 s4 = _mm_aesenc_si128(s4, rk);
8 s5 = _mm_aesenc_si128(s5, rk);
9 s6 = _mm_aesenc_si128(s6, rk);
10 s7 = _mm_aesenc_si128(s7, rk);
```

多分组接口按照 8 路、4 路、1 路的顺序处理输入：

Listing 15: AES-NI 8/4/1 路调度策略

```
1 while (blocks >= 8U) {
2     encrypt_eight_blocks_aesni(
3         ctx, in, out);
4
5     in      += 8U * AES128_BLOCK_SIZE;
```

```
6     out    += 8U * AES128_BLOCK_SIZE;
7     blocks -= 8U;
8 }
9
10 if (blocks >= 4U) {
11     encrypt_four_blocks_aesni(
12         ctx, in, out);
13
14     in      += 4U * AES128_BLOCK_SIZE;
15     out     += 4U * AES128_BLOCK_SIZE;
16     blocks -= 4U;
17 }
18
19 while (blocks != 0U) {
20     encrypt_one_block_aesni(
21         ctx, in, out);
22
23     in  += AES128_BLOCK_SIZE;
24     out += AES128_BLOCK_SIZE;
25     --blocks;
26 }
```

该调度方式能够在较长输入中提高独立状态之间的并行度，同时避免中等长度消息全部退化为单分组调用。

3.3.3 运行时检测与回退

程序通过 `__builtin_cpu_supports("aes")` 检测 AES-NI 支持情况。实际回退策略为：

- AES-NI 加密接口在硬件不支持时回退到 T-table 加密；
- AES-NI 解密接口在硬件不支持时回退到字节级解密；
- AVX2 加密接口在硬件不支持或剩余分组不足 8 个时回退到 T-table 加密。

这种回退设计保证程序能够在不支持相应指令集的处理器上继续运行，同时避免触发非法指令。

4 CTR 工作模式实现与优化

4.1 模式原理

CTR 模式将计数器分组加密为密钥流，再与输入异或：

$$C_i = P_i \oplus E_K(\text{CTR}_i), \quad P_i = C_i \oplus E_K(\text{CTR}_i). \quad (6)$$

各计数器分组之间不存在数据依赖，因此适合调用多分组 AES 后端。本实现将计数器视为 128 bit 大端整数，并在处理前检查计数器是否会发生回绕。

4.2 实现与优化

Reference 和 T-table 版本逐分组生成密钥流；AVX2 版本一次构造 8 个连续计数器，再调用八分组加密后端。AES-NI 版本进一步使用 `pshufb` 完成低 32 bit 计数器的字节序转换，并采用 8/4/1 路处理。

AES-NI 八路核心如下：

Listing 16: CTR AES-NI 八路处理核心

```
1  const uint32_t low = load_be32(counter + 12U);
2
3  if (low <= UINT32_MAX - 7U) {
4      make_counter_batch_x86(
5          counter, counter_blocks, 8U
6      );
7
8      aes128_encrypt_blocks_aesni(
9          ctx, counter_blocks, keystream, 8U
10     );
11
12     xor_full_blocks_x86(
13         in, keystream, out, 8U
14     );
15
16     counter_add_be128(counter, 8U);
17 }
```

当低 32 bit 接近进位边界时，程序暂时逐分组处理；剩余 4 个分组使用四路 AES-NI，非整分组尾部使用字节异或。硬件路径不可用时回退到 T-table 批量实现。

5 GCM 工作模式实现与优化

5.1 模式原理

GCM 由 GCTR 加密和 GHASH 认证组成。哈希子密钥为

$$H = E_K(0^{128}), \quad (7)$$

认证标签为

$$T = \text{MSB}_t(E_K(J_0) \oplus \text{GHASH}_H(A, C)). \quad (8)$$

对于 96 bit IV, 直接构造 $J_0 = IV \parallel 0^{31} \parallel 1$; 其他长度的 IV 通过 GHASH 计算。

5.2 后端设计

GCM 将 AES 加密和 $GF(2^{128})$ 乘法分别抽象为后端:

Listing 17: GCM 后端结构

```
1 typedef struct {
2     encrypt_block_fn  encrypt_block;
3     encrypt_blocks_fn encrypt_blocks;
4     ghash_multiply_fn multiply;
5 } gcm_backend_t;
```

实验比较以下三种组合:

1. Reference AES 与便携式 GHASH;
2. AES-NI 与便携式 GHASH;
3. AES-NI 与 PCLMULQDQ GHASH。

便携式 GHASH 逐位完成有限域乘法, 并使用掩码代替数据相关分支。PCLMULQDQ 版本计算四个 64 bit 无进位乘积:

Listing 18: PCLMULQDQ 乘法核心

```
1 const __m128i p00 =
2     _mm_clmulepi64_si128(a, b, 0x00);
3 const __m128i p01 =
4     _mm_clmulepi64_si128(a, b, 0x01);
5 const __m128i p10 =
6     _mm_clmulepi64_si128(a, b, 0x10);
```

```
7 const __m128i p11 =  
8     _mm_clmulepi64_si128(a, b, 0x11);  
9  
10 const __m128i cross =  
11     _mm_xor_si128(p01, p10);
```

乘积组合为 256 bit 多项式后，再按 $x^{128} + x^7 + x^2 + x + 1$ 约减。输入输出使用 `pshufb` 完成逐字节比特顺序转换。

加密时先执行 GCTR，再对 AAD 和密文计算 GHASH。解密时先重新计算并比较标签，只有验证成功后才输出明文。

6 XTS 工作模式实现与优化

6.1 模式原理

XTS 使用两个独立的 AES-128 密钥。数据密钥 K_1 用于加解密，tweak 密钥 K_2 用于生成初始 tweak：

$$T_0 = E_{K_2}(I), \quad T_j = \alpha^j T_0. \quad (9)$$

第 j 个分组的加密为

$$C_j = E_{K_1}(P_j \oplus T_j) \oplus T_j. \quad (10)$$

6.2 tweak 更新

实现中 tweak 按小端字节序表示。乘以 α 等价于整体左移一位，若产生进位，则在最低字节异或 0x87：

Listing 19: XTS tweak 更新

```
1 static void xts_mul_alpha(uint8_t tweak[16]) {  
2     uint8_t carry = 0U;  
3  
4     for (size_t i = 0; i < 16U; ++i) {  
5         const uint8_t next = tweak[i] >> 7U;  
6         tweak[i] =  
7             (uint8_t)((tweak[i] << 1U) | carry);  
8         carry = next;  
9     }  
10 }
```

```
11     if (carry != 0U) {  
12         tweak[0] ^= UINT8_C(0x87);  
13     }  
14 }
```

6.3 批量处理

批量版本仍顺序生成连续 tweak，但将 8 个或 4 个 tweak 集中保存，随后统一完成输入白化、多分组 AES 和输出白化：

Listing 20: XTS 八分组批量处理

```
1 make_tweak_batch(tweak, tweaks, 8U);  
2  
3 xor_batch(  
4     in, tweaks, temporary, 8U  
5 );  
6  
7 crypt(  
8     data_key, temporary, temporary, 8U  
9 );  
10  
11 xor_batch(  
12     temporary, tweaks, out, 8U  
13 );
```

其中，xor_batch 在 x86 平台上使用 _mm_xor_si128，crypt 在加密时指向 AES-NI 加密后端，在解密时指向 AES-NI 解密后端。

对于非 16 字节整数倍的数据，程序使用 ciphertext stealing：最后一个完整分组先产生临时密文，其前若干字节作为尾部密文，剩余部分与尾部明文重新组成完整分组，再使用下一个 tweak 加密。该方法保持密文长度与明文长度一致。

7 实验验证与性能测试设计

7.1 AES 核心正确性验证

AES 核心分别使用 FIPS-197 和 SP 800-38A 标准向量进行验证，并通过随机密钥、多分组交叉验证和原地加密测试。T-table 完成 4096 组密钥、每组 8 个分组的交叉验证；AVX2 实现完成 2048 组变长数据交叉验证。

```
100% tests passed, 0 tests failed out of 2

Total Test time (real) = 0.02 sec
[PASS] FIPS-197 AES-128
[PASS] SP 800-38A AES-128
[PASS] 8-block roundtrip
All AES-128 reference tests passed.
[PASS] T-table FIPS-197 vector
[PASS] 4096 keys x 8 blocks reference/T-table cross-check
[PASS] T-table in-place encryption
All AES-128 T-table tests passed.
AES-128 reference vs four-table benchmark
Bytes      Ref MiB/s    T MiB/s      Ref c/B      T c/B      Speedup
16         127.43    316.77       23.853       9.596       2.49x
64         129.91    324.28       23.397       9.373       2.50x
256        131.83    302.30       23.056       10.055      2.29x
1024        136.36    332.08       22.291       9.153       2.44x
8192        131.75    306.98       23.070       9.901       2.33x
65536       128.81    312.57       23.596       9.724       2.43x
1048576     123.92    307.02       24.529       9.980       2.46x
benchmark checksum: 0
```

(a) T-table 测试结果

```
All AES-128 AVX2 gather/shuffle tests passed.
benchmark checksum: 1199102176
AVX2 hardware path: enabled
AES-128 reference vs T-table vs AVX2 gather/shuffle
Bytes      Ref MiB/s    T MiB/s    AVX MiB/s    Ref c/B      T c/B      AVX c/B      T speed  AVX speed
16         131.89    301.89     300.88       23.047       10.069     10.102       2.29x    2.28x
64         136.45    319.17     324.35       22.276       9.523       9.371       2.34x    2.38x
128        135.18    326.68     733.57       22.486       9.304       4.144       2.42x    5.43x
256        132.51    319.96     721.24       22.766       9.580       4.215       2.40x    5.40x
1024        134.15    328.58     670.46       22.657       9.250       4.534       2.45x    5.00x
8192        142.76    316.73     675.66       21.292       9.597       4.499       2.22x    4.73x
65536       134.12    316.96     712.88       22.663       9.590       4.264       2.36x    5.32x
1048576     134.56    322.27     677.89       22.588       9.432       4.484       2.39x    5.04x
(.venv) wem@localhost:~/aes128-optimization$
```

(b) AVX2 测试结果

图 2: AES 核心实现的正确性验证

7.2 工作模式正确性验证

CTR 通过 NIST SP 800-38A 标准向量、任意长度交叉验证、计数器进位和耗尽测试；GCM 通过 SP 800-38D 向量，并验证 AAD、非 96 bit IV、截断 Tag 和篡改拒绝；XTS 通过完整分组、ciphertext stealing 及原地加解密测试。

```
All AES-128 AES-NI tests passed.
[INFO] CTR AES-NI batch path: enabled
[PASS] NIST SP 800-38A AES-128-CTR vector on all backends
[PASS] 2048 arbitrary-length CTR cross-checks (including partial blocks)
[PASS] CTR in-place operation and low-32-bit counter carry
[PASS] CTR counter-exhaustion guard and zero-length handling
All AES-128-CTR tests passed.
[INFO] GCM PCLMULQDQ path: enabled
[PASS] NIST SP 800-38D AES-128-GCM vectors (96-bit and non-96-bit IVs)
[PASS] 1024 arbitrary-length GCM cross-checks with AAD, IV and truncated tags
[PASS] GCM in-place operation, constant-time tag check and tamper rejection
All AES-128-GCM tests passed.
benchmark checksum: 451571975
AES-NI hardware path: enabled
GCM PCLMULQDQ path: enabled
AES-128-GCM reference vs AES-NI/portable-GHASH vs AES-NI/PCLMUL
Bytes      Ref MiB    NI-GH MiB    PCL MiB    Ref c/B    NI-GH c/B    PCL c/B    NI-GH x    PCL x
16         13.4      19.4         79.0      227.288    156.566     38.485     1.45x     5.91x
64         32.0      46.2        220.3      95.066     64.701     13.705     1.47x     6.83x
128        40.5      64.8        336.1      75.100     46.907     9.044      1.60x     8.38x
256        45.8      73.7        412.4      66.390     41.255     7.370      1.61x     9.01x
1024       51.5      86.4        515.4      59.085     35.189     5.897      1.68x    10.01x
8192       53.4      88.4        542.3      56.944     34.398     5.605     1.66x    10.16x
65536      53.3      84.8        531.9      57.026     35.862     5.714     1.59x     9.98x
1048576    51.5      88.0        524.3      58.994     34.558     5.798     1.71x    10.15x
```

(a) CTR 与 GCM 测试结果

```
All AES-128 XTS tests passed.
benchmark checksum: 3306077978
XTS AES-NI batch path: enabled
AES-128-XTS reference vs AES-NI scalar vs AES-NI batch-tweak
Bytes      Ref MiB    NI-Sc MiB    NI-Bat MiB    Ref c/B    NI-Sc c/B    NI-Bat c/B    NI-Sc x    NI-Bat x
16         63.5      723.6       697.4       47.847      4.143       4.359       11.55x    10.98x
64        99.7      552.6       421.5       30.499      5.501       2.138       5.54x    14.26x
128       108.6      804.3       1258.3      27.980      3.779       2.416       7.40x    11.58x
256       118.7      690.1       1186.1      25.608      4.405       2.563       5.81x     9.99x
1024      121.7      800.9       1536.6      24.971      3.795       1.978       6.58x    12.62x
8192      124.5      791.0       1498.4      24.408      3.843       2.029       6.35x    12.03x
65536     122.5      762.8       1496.6      24.806      3.985       2.031       6.23x    12.21x
1048576   120.9      727.3       1423.1      25.140      4.179       2.136       6.02x    11.77x
(.venv) wem@localhost:~/aes128-optimization$
```

(b) XTS 测试结果

图 3: 工作模式正确性验证

7.3 硬件指令验证

反汇编结果中可以观察到 AES-NI 加解密指令，以及 GCM 使用的 PCLMULQDQ 和 pshufb 指令，说明相应硬件路径已实际生成。

```
(.venv) wem@localhost:~/aes128-optimization$ ./tools/check_aesni_asm.sh | tee phase4_aesni_asm.txt
[aesenc]
20: 66 0f 38 dc c3      aesenc xmm0,xmm3
2f: 66 0f 38 dc c4      aesenc xmm0,xmm4
41: 66 0f 38 dc c5      aesenc xmm0,xmm5
[aesenclast]
74: 66 0f 38 dd c1      aesenclast xmm0,xmm1
196: 66 0f 38 dd dc     aesenclast xmm3,xmm4
19b: 66 0f 38 dd d4      aesenclast xmm2,xmm4
[aesdec]
489: 66 0f 38 de c1      aesdec xmm0,xmm1
49d: 66 0f 38 de c1      aesdec xmm0,xmm1
4ac: 66 0f 38 de c1      aesdec xmm0,xmm1
[aesdeclast]
4f1: 66 0f 38 df c1      aesdeclast xmm0,xmm1
643: 66 0f 38 df dc     aesdeclast xmm3,xmm4
648: 66 0f 38 df d4      aesdeclast xmm2,xmm4
[aesimc]
471: 66 0f 38 db ca      aesimc xmm1,xmm2
48e: 66 0f 38 db cb      aesimc xmm1,xmm3
4a2: 66 0f 38 db cd      aesimc xmm1,xmm5
```

(a) AES-NI 指令

```
(.venv) wem@localhost:~/aes128-optimization$ ./tools/check_gcm_asm.sh | tee phase6_gcm_asm.txt
[v7pclmul[a-z]*qdq]
9d8: 66 0f 3a 44 f2 01    pclmulqlqdq xmm6,xmm2
9de: 66 0f 3a 44 ea 10    pclmulqlqdq xmm5,xmm2
9e9: 66 0f 3a 44 ca 00    pclmulqlqdq xmm1,xmm2
9f3: 66 0f 3a 44 c2 11    pclmulqlqdq xmm6,xmm2
[v7pshufb]
950: 66 0f 38 00 c2      pshufb xmm0,xmm2
970: 66 0f 38 00 ea      pshufb xmm5,xmm2
994: 66 0f 38 00 d1      pshufb xmm2,xmm1
9bd: 66 0f 38 00 e9      pshufb xmm5,xmm1
[AES-NI round instructions are verified separately by tools/check_aesni_asm.sh]
```

(b) PCLMULQDQ 与 shuffle 指令

图 4: 关键硬件指令的反汇编验证

7.4 实验环境

表 2: 实验环境

项目	配置
操作系统	Ubuntu 24.04.4 LTS (WSL2)
处理器	13th Gen Intel Core i5-13500H
编译器	GCC 13.3.0
构建配置	CMake Release, -O3 -DNDEBUG
指令集	AES-NI、AVX2、PCLMULQDQ、SSSE3、SSE4.1
CPU 绑定	逻辑 CPU 0
测试长度	16 B 至 1 MiB, 共 8 组
测试策略	预热 2 次, 运行 10 次并取中位数
性能指标	MiB/s、cycles/byte、相对加速比

7.5 性能测试设计

为保证不同实现之间具有可比性, 各版本在同一基准测试程序中运行, 使用相同的密钥、输入数据、缓冲区长度、编译配置和 CPU 核心。测试对象包括:

- AES 核心: Reference、T-table、AVX2 和 AES-NI;
- CTR: Reference、T-table、AVX2 和 AES-NI 批量版本;
- GCM: Reference、AES-NI 与便携式 GHASH、AES-NI 与 PCLMUL;
- XTS: Reference、AES-NI 标量和 AES-NI 批量 tweak 版本。

测试长度设置为 16 B、64 B、128 B、256 B、1 KiB、8 KiB、64 KiB 和 1 MiB。短消息用于观察函数调用、状态初始化和尾部处理等固定开销; 128 B 正好包含 8 个 AES 分组, 可用于观察八路并行路径的性能变化; 较长消息用于评价实现进入稳定状态后的持续吞吐率。

正式计时前, 每个基准程序先运行 2 次作为预热, 使代码和数据进入缓存, 并减小首次执行带来的额外开销。正式测试使用 `taskset -c 0` 绑定逻辑 CPU 0, 避免线程在不同逻辑 CPU 之间迁移。

为防止编译器将未使用的加密结果删除, 基准程序对输出数据计算并打印 checksum。所有实现均在 Release 模式下编译, 并在同一可执行程序中进行比较。

7.6 性能指标与统计方法

吞吐量按照处理字节数与运行时间计算：

$$\text{Throughput} = \frac{\text{Bytes}}{\text{Time} \times 2^{20}} \quad \text{MiB/s.} \quad (11)$$

单位字节周期数定义为：

$$\text{cycles/byte} = \frac{\text{CPU cycles}}{\text{Bytes}}. \quad (12)$$

相对加速比以对应模块的 Reference 版本为基准：

$$\text{Speedup} = \frac{\text{Reference cycles/byte}}{\text{Optimized cycles/byte}}. \quad (13)$$

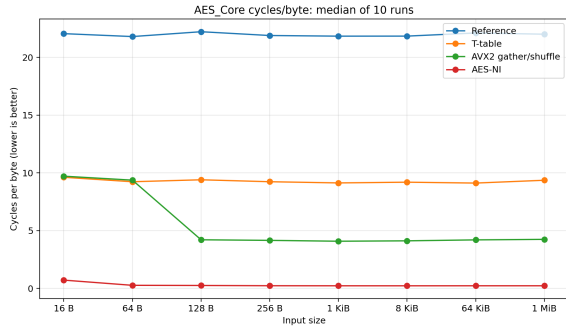
每个基准程序正式运行 10 次，并分别保存原始输出。最终结果不是取单次最佳值，而是对每种输入长度和每项性能指标逐列计算中位数。使用中位数可以减小 WSL 调度、后台进程和处理器频率波动造成的偶然离群值影响。

8 实验结果与分析

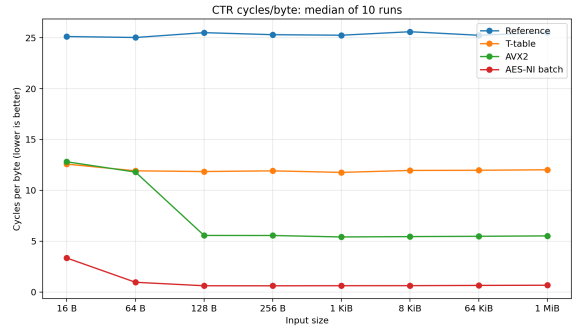
8.1 1 MiB 输入下的总体结果

表 3: 1 MiB 输入下的 10 次测试中位数

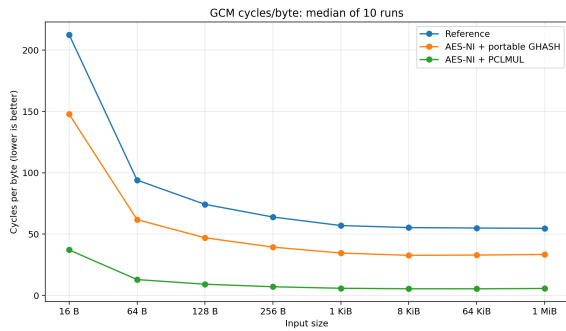
模块	最佳优化版本	Ref MiB/s	Opt MiB/s	Ref c/B	Opt c/B	加速比
AES 核心	AES-NI	138.10	13125.80	22.0040	0.2315	94.995×
CTR	AES-NI batch	119.10	4527.45	25.5195	0.6715	38.255×
GCM	AES-NI + PCLMUL	55.55	531.75	54.7295	5.7160	9.575×
XTS	AES-NI batch tweak	116.80	1451.40	26.0155	2.0945	12.530×



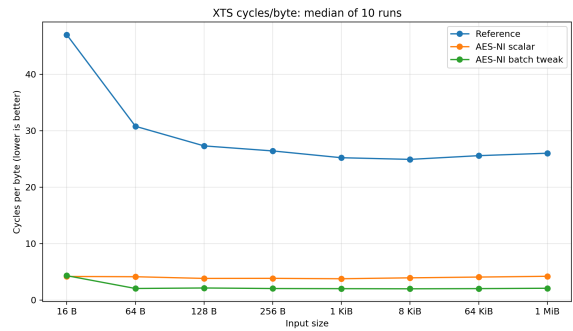
(a) AES 核心



(b) CTR 模式



(c) GCM 模式



(d) XTS 模式

图 5: 不同输入长度下各实现的单位字节周期数

8.2 AES 核心性能分析

T-table 将 Reference 的约 22.00 cycles/byte 降至 9.36 cycles/byte, 在 1 MiB 下获得 2.37 倍加速。AVX2 八路实现进一步降至 4.25 cycles/byte, 获得 5.21 倍加速。AES-NI 在 1 MiB 下达到 0.2315 cycles/byte 和 13125.8 MiB/s, 相对教学型 Reference 获得约 95 倍加速。

16 B 时 AES-NI 仅使用单分组路径, 固定调用开销占比较高; 64 B 可使用四路交错; 128 B 起使用八路交错, 因此性能在 64 B 至 128 B 之间出现明显跃升。需要强调, 95 倍是相对于本实验结构清晰但未深度优化的 Reference, 而不是对所有软件 AES 实现的普遍结论。

8.3 CTR 性能分析

CTR 在 AES 运算之外还需要计数器构造、递增、字节序转换和密钥流异或, 因此 AES-NI CTR 的单位字节开销高于裸 AES。1 MiB 下, Reference、T-table、AVX2 和 AES-NI batch 分别为 25.5195、12.0225、5.5210 和 0.6715 cycles/byte。AES-NI batch 吞吐率为 4527.45 MiB/s, 相对 Reference 加速 38.255 倍。

128 B 正好包含 8 个 AES 分组, AVX2 与 AES-NI 八路路径从该长度起充分发挥作用。较长消息中 AES-NI CTR 稳定在约 0.62–0.67 cycles/byte, 说明模式层额外开销

约为裸 AES 的数倍，但仍保持多 GiB/s 吞吐率。

8.4 GCM 性能分析

仅将 GCTR 替换为 AES-NI 后，1 MiB 性能由 54.7295 cycles/byte 降至 33.4260 cycles/byte，整体仅加速 1.64 倍。这表明 AES 部分加速后，逐位实现的 GHASH 成为主要瓶颈。使用 PCLMULQDQ 后，性能进一步降至 5.7160 cycles/byte，吞吐率达到 531.75 MiB/s，相对 Reference 加速 9.575 倍，相对相同 AES-NI 后端下的普通 GHASH 约加速

$$\frac{33.4260}{5.7160} \approx 5.85. \quad (14)$$

短消息需要承担 H 、 J_0 、长度块和 Tag 生成等固定开销，因此 16 B 时单位字节成本较高；1 KiB 以后 PCLMUL 路径稳定在约 5.4–5.9 cycles/byte。

8.5 XTS 性能分析

1 MiB 下，AES-NI 标量 XTS 将性能由 26.0155 cycles/byte 降至 4.2135 cycles/byte，获得 6.16 倍加速。批量 tweak 与 8/4/1 路 AES-NI 将性能进一步降至 2.0945 cycles/byte，吞吐率达到 1451.40 MiB/s，相对 Reference 加速 12.53 倍，相对 AES-NI 标量约提升

$$\frac{4.2135}{2.0945} \approx 2.01. \quad (15)$$

16 B 只有一个分组，批量版本不能利用多路并行且存在额外调度，因此略慢于 AES-NI 标量；64 B 及以上开始体现批量 tweak 和多状态交错的优势。1 KiB 至 1 MiB 范围内批量版本稳定在约 2.0–2.1 cycles/byte。

9 安全性与工程权衡

9.1 T-table 与缓存侧信道

T-table 和 AVX2 gather 实现的访存地址由秘密中间状态决定。攻击者若能观测缓存命中、驱逐或执行时间，理论上可能恢复与密钥相关的信息。因此，这两类实现适合用于理解查表与 SIMD 优化，不适合直接作为高威胁环境中的密码库后端。AES-NI 不依赖秘密相关表地址，在本实验涉及的缓存访问模型下具有更好的安全属性。

9.2 常数时间与认证失败处理

便携式 GHASH 使用掩码选择避免秘密相关分支；PCLMUL 版本主要由固定指令序列组成。GCM Tag 采用累积异或方式比较，不在第一个不同字节处提前返回。解密

只有在 Tag 验证通过后才执行 GCTR 并写出明文，从而避免释放未经认证的数据。

9.3 输入边界与状态管理

CTR 在处理前计算所需分组数，并拒绝会使 128 bit 计数器回绕的请求。GCM 限制 GCTR 分组数量不超过 $2^{32} - 2$ ，并检查字节长度转换为 bit 长度时的溢出。XTS 要求数据单元至少包含一个完整分组，限制最多 2^{20} 个分组，并拒绝两个 AES-128 子密钥完全相同的弱组合。临时密钥流、GHASH 中间值、tweak 数组和扩展密钥均在使用后清零。

9.4 实验实现的局限性

本实验目标是展示算法与指令集优化过程，而不是替代成熟密码库，主要局限包括：

1. Reference、T-table 与 AVX2 路径未达到生产密码库的所有安全加固要求；
2. PCLMUL GHASH 采用易于讲解的单块乘法，没有实现 OpenSSL 等库中的多块聚合与深流水；
3. XTS tweak 乘 α 仍采用可读性较好的标量代码，未做更深度向量化；
4. 性能数据来自单台 WSL 环境，绝对数值不能直接代表其他微架构；
5. 本实验验证的是算法一致性和常见内存错误，不等价于密码模块认证。

10 结论

本实验完成了 AES-128 从参考实现到通用 SIMD 和专用指令的完整优化链路，并进一步实现了 CTR、GCM 与 XTS 三种工作模式。实验结果表明：

1. T-table 通过合并轮变换获得约 2.37 倍加速，但带来缓存侧信道风险；
2. AVX2 gather/shuffle 八路实现将 1 MiB AES 性能提升至 Reference 的约 5.21 倍；
3. AES-NI 直接执行轮变换，在 1 MiB 裸 AES 中达到约 0.2315 cycles/byte；
4. CTR 能充分利用计数器间的独立性，AES-NI batch 相对 Reference 加速约 38.26 倍；
5. GCM 中仅优化 AES 会使 GHASH 成为瓶颈，PCLMULQDQ 将整体加速提高到约 9.58 倍；

6. XTS 的批量 tweak 预计算与多分组 AES-NI 相对 Reference 获得约 12.53 倍加速；
7. 10 次固定核心测试的中位数结果显示，各优化路径在较长输入下具有稳定且可解释的性能趋势。

实验验证了一个重要的软件优化规律：对称密码性能不仅取决于算法指令数，还取决于数据布局、寄存器利用率、分组间并行度、缓存行为和模式层附加运算。专用指令可以显著加速核心算法，但只有同时识别并优化 CTR 计数器、GCM 的 GHASH 和 XTS 的 tweak 等模式瓶颈，才能获得完整系统层面的性能提升。

A 完整性能中位数数据

A.1 AES 核心

表 4: AES 核心 10 次中位数 (cycles/byte 与加速比)

Bytes	Ref c/B	T c/B	AVX2 c/B	NI c/B	T speedup	AVX2 speedup	NI speedup
16	22.0515	9.6220	9.7155	0.7210	2.290	2.280	30.695
64	21.7995	9.2355	9.3735	0.2710	2.345	2.335	80.965
128	22.2180	9.4045	4.2120	0.2580	2.370	5.170	85.975
256	21.8900	9.2365	4.1610	0.2355	2.400	5.305	92.550
1024	21.8345	9.1330	4.0870	0.2305	2.395	5.355	94.985
8192	21.8380	9.1960	4.1215	0.2295	2.375	5.260	94.935
65536	22.1105	9.1200	4.2035	0.2315	2.420	5.300	95.395
1048576	22.0040	9.3635	4.2485	0.2315	2.370	5.210	94.995

A.2 CTR

表 5: CTR 10 次中位数 (cycles/byte 与加速比)

Bytes	Ref c/B	T c/B	AVX2 c/B	NI c/B	T speedup	AVX2 speedup	NI speedup
16	25.1205	12.5785	12.8060	3.3475	1.990	1.940	7.555
64	25.0230	11.9180	11.7955	0.9620	2.095	2.120	26.145
128	25.5005	11.8485	5.5655	0.6200	2.125	4.550	40.760
256	25.2995	11.9170	5.5600	0.6155	2.145	4.600	40.850
1024	25.2475	11.7630	5.4170	0.6245	2.155	4.735	41.195
8192	25.5910	11.9550	5.4520	0.6290	2.130	4.695	40.495
65536	25.2425	11.9710	5.4820	0.6535	2.135	4.700	38.325
1048576	25.5195	12.0225	5.5210	0.6715	2.120	4.705	38.255

A.3 GCM

表 6: GCM 10 次中位数 (cycles/byte 与加速比)

Bytes	Ref c/B	NI+GHASH c/B	PCLMUL c/B	NI+GHASH speedup	PCLMUL speedup
16	212.6085	147.8405	37.1345	1.430	5.720
64	94.0595	61.8350	12.9755	1.525	7.210
128	74.2995	47.0465	9.1780	1.565	8.005
256	63.9705	39.4295	7.1165	1.625	9.005
1024	57.0200	34.6000	5.8685	1.640	9.745
8192	55.3455	32.7255	5.4650	1.680	10.135
65536	55.0040	32.9475	5.4410	1.660	9.980
1048576	54.7295	33.4260	5.7160	1.640	9.575

A.4 XTS

表 7: XTS 10 次中位数 (cycles/byte 与加速比)

Bytes	Ref c/B	NI scalar c/B	NI batch c/B	NI scalar speedup	NI batch speedup
16	47.0015	4.1865	4.3225	11.170	10.975
64	30.7855	4.1380	2.0550	7.460	14.935
128	27.3105	3.8365	2.1430	7.075	12.805
256	26.4190	3.8485	2.0480	6.850	12.975
1024	25.2160	3.7805	2.0230	6.640	12.495
8192	24.9215	3.9420	1.9990	6.360	12.575
65536	25.5805	4.0780	2.0315	6.240	12.395
1048576	26.0155	4.2135	2.0945	6.160	12.530